



DISRUPTORS MEDIA · FIELD GUIDE

Scrape The Web For *Free.*

A plain-English playbook for ScrapeGraph AI — the open-source library that turns any website into clean, structured JSON without CSS selectors, XPath, or brittle scripts. Setup, code, pipelines, and the agency business model on top of it.

\$0

MIT-LICENSED TOOL COST

10 min

FROM ZERO TO FIRST SCRAPE

\$2–5K

PER SCRAPE JOB (AGENCY
RATE)



SECTION 01

Web scraping is the 90%. This guide automates it.

Scraping is the slow, repetitive, technical work that eats your day — copy-pasting from LinkedIn, watching prices on Amazon, pulling agent contacts off Zillow, building lead lists by hand. The right tool turns weeks of work into a single prompt. This is that tool.

Meet ScrapeGraph AI

ScrapeGraph AI is a **free, MIT-licensed Python library** that uses an LLM to scrape any website with a single plain-English prompt. No CSS selectors. No XPath. No brittle rules that break when a site updates. You describe what you want; the model figures out how to extract it.

github.com/ScrapeGraphAI/Scrapegraph-ai

Open source. MIT licensed. Active community. The library is free to use forever.

Pluggable LLM backend

Drop in OpenAI, Anthropic, or run a local model with Ollama. Pay pennies, or pay nothing.

What makes it different

OLD WAY

Hand-write CSS selectors and XPath

Breaks every time the site changes its layout.

BeautifulSoup, Scrapy, Selenium glue code

Hours of setup and constant maintenance.

\$50–200 / mo SaaS subscriptions

Per-row pricing on every paid scraping platform.

SCRAPEGRAPH AI

Write a prompt in English

"Get every product name, price, and rating from this page."

One function call

The LLM reads the page and returns structured JSON.

Pennies in inference, or \$0 with Ollama

You own the pipeline end-to-end.

THE PROMISE

Spend 10 minutes on setup. Trade weeks of manual data gathering for a script that runs on demand and outputs clean, structured JSON every time.

The rest of this guide walks you through installation, picking your LLM, running your first scrape, and a workflow for getting AI to write the script for you. We close with the use cases this unlocks — including the \$2–5K-per-job lead-gen model operators are already running.



SECTION 02

Install the library. Pick your model.

Two steps stand between you and a working scraper. Total time on a clean machine: ten minutes.

1

Install ScrapeGraph AI

TERMINAL • ~2 MINUTES

Open your terminal and run these two commands. Playwright handles the browser layer that lets the scraper render JavaScript-heavy sites.

```
# Install the library and its browser engine
pip install scrapegraphai
playwright install
```

RECOMMENDED

Run this inside a Python virtual environment (`python -m venv venv`) to keep dependencies isolated from the rest of your system.

2

Choose your LLM backend

HOSTED API OR LOCAL MODEL

ScrapeGraph AI is model-agnostic. Pick the lane that fits your speed, privacy, and budget needs.

OPTION	WHERE	COST	BEST FOR
OpenAI RECOMMENDED	platform.openai.com	Pennies per scrape	Fastest setup. Best accuracy on messy pages. GPT-4o-mini is the sweet spot.
Anthropic	console.anthropic.com	Pennies per scrape	Long context, strong reasoning. Great for multi-page extraction.
Ollama FREE	ollama.com	\$0 — runs locally	Total privacy. Zero ongoing cost. Slower; quality varies by model.

IF YOU GO LOCAL WITH OLLAMA

Install Ollama from ollama.com, then pull a model. Llama 3.2 is a solid starting point that balances speed and quality on consumer hardware.

```
# Pull a local model — runs entirely on your machine
ollama pull llama3.2
```



SECTION 03

Run your first scrape.

The same library, two flavors. Copy the snippet that matches the model you picked, swap the URL and the prompt for your target, and run it.

3

Hosted API — OpenAI GPT-4o-mini

BEST ACCURACY • PENNIES PER RUN

Paste your API key, point at any public URL, write a prompt that names the fields you want, and run the script. The output is a Python dict you can save as JSON.

```
from scrapegraphai.graphs import SmartScraperGraph

graph_config = {
    "llm": {"api_key": "YOUR_OPENAI_KEY", "model": "openai/gpt-4o-mini"},
    "verbose": True, "headless": True,
}

scraper = SmartScraperGraph(
    prompt="List every product name, price, and rating.",
    source="https://example.com/products",
    config=graph_config,
)
print(scraper.run())
```

3b

Local model — Ollama (Llama 3.2)

\$0 ONGOING • STAYS ON YOUR MACHINE

Same shape, no API key. Make sure `ollama serve` is running in another terminal before you launch the scraper.

```
from scrapegraphai.graphs import SmartScraperGraph

graph_config = {
    "llm": {
        "model": "ollama/llama3.2", "temperature": 0,
        "format": "json", "model_tokens": 8192,
        "base_url": "http://localhost:11434",
    },
    "verbose": True, "headless": True,
}

scraper = SmartScraperGraph(
    prompt="Extract every job title, company, and location.",
    source="https://example.com/jobs",
    config=graph_config,
)
print(scraper.run())
```

CHECKPOINT

Run the script. You should see a printed JSON-shaped Python dict in your terminal. If you do, you're done — every future scrape is a variation on this same eight-line pattern.



SECTION 04

You don't need to know Python.

You need to know **what data you want**.

Step 4 is the unlock. Hand the work off to Claude Code or OpenAI Codex. Describe the data, paste in the target URL, and ship a working scraper without writing a line yourself.

4

Prompt your AI assistant

CLAUDE CODE • CODEX • CURSOR

Open your AI coding assistant and paste a prompt like the one below. Swap in your target URL and the exact fields you need. The assistant generates a runnable script you can execute immediately.

```
# Paste into Claude Code or Codex
```

```
Using the ScrapeGraphAI Python library with OpenAI GPT-4o-mini,
write a script that scrapes [TARGET URL] and returns
a JSON list of every [ENTITY] with the following fields:
[FIELD 1], [FIELD 2], [FIELD 3].
```

```
Save the output to scrape_output.json.
```

THE INSIGHT

The bottleneck for most operators is not Python skill — it's knowing what to ask for. Once you can describe the data clearly, an AI assistant writes the scraper.

The pipelines under the hood

ScrapeGraph AI ships with five graph types. **For 90% of jobs, SmartScraperGraph is all you need.** Reach for the others when you scale up.

PIPELINE	USE IT FOR
SmartScraperGraph DEFAULT	Single page, one prompt. Your daily driver.
SmartScraperMultiGraph	A list of URLs scraped in parallel — same prompt, many pages.
SearchGraph	Run a search query, then scrape the top N results in one call.
ScriptCreatorGraph	Generates a reusable, standalone Python script for the target site.
SpeechGraph	Extract data and return the result as synthesized audio output.



SECTION 05

The targets. The business. The fine print.

What you can scrape — in plain English

LinkedIn

Names, titles, companies, bios for outbound lists.

Amazon

Pricing, reviews, specs for competitive intel.

Zillow

Property data and agent contacts for real-estate ops.

Glassdoor

Ratings and reviews for employer-brand research.

Twitter / X

Handles, bios, follower counts for creator outreach.

Email Directories

Bulk contacts for verified outreach campaigns.

Multi-Page Datasets

Crawl paginated lists in a single parallel run.

Custom Sources

Any public site — the prompt is the spec.

The agency model on top of this

Lead-gen operators are charging **\$2,000–\$5,000 per scrape job** using this exact workflow. One laptop. Zero platform fees. The margin is almost entirely yours.

01 • INTAKE**Client brief**

Industry, geography, job title, platform.

02 • RUN**Execute scrape**

Point ScrapeGraph AI at the source list.

03 • DELIVER**Structured JSON**

Hand back clean data within hours, not weeks.

04 • INVOICE**Bill the job**

Flat fee per project. No SaaS markup.

The fine print — what to expect

- **The tool is free, inference is not.** OpenAI and Anthropic charge pennies per scrape. Budget accordingly on large jobs.
- **Ollama is \$0 but variable.** Local models are slower and quality depends on your hardware and chosen model.
- **No GUI.** Everything runs from Python or an AI code assistant — there is no point-and-click interface.
- **Scale needs proxies.** Heavy-volume scraping triggers rate limits. Add a basic proxy rotation layer once you cross thousands of requests per run.

THE TAKEAWAY

Web scraping just got commoditized. The leverage is no longer in writing brittle code — it's in knowing what data is worth pulling, and turning it into a deliverable a client will pay for.